
TRABALHO PRÁTICO 1 – AC III

Controlador de Cache

Sumário

1. Introdução	3
2. Revisão Teórica	3
2.1 Hierarquia de Memória	3
2.2 Funcionamento da Cache	4
2.3 Políticas de Leitura e Escrita	4
2.4 Tratamento de Misses	5
3. Arquitetura do Projeto	5
3.1 Organização dos Endereços	5
3.2 Estrutura Interna da Cache	6
3.3 Máquina de Estados (FSM)	6
3.4 Interface com o Processador e Memória	7
4. Decisões de Projeto	8
5. Implementação em SystemVerilog	9
5.1 Módulo Principal: cache_controller.sv	9
5.2 Testbench: tb_cache_controller.sv	12
6. Resultados e Validação Funcional	16
6.1 Testes de Leitura	16
6.2 Testes de Escrita	17
6.3 Testes de Substituição	17
6.4 Testes de Consistência	18
6.5 Testes de Casos Limite	18
7. Instruções de Compilação e Simulação	19
8. Uso de Inteligência Artificial	20

9. Conclusão	22
10. Referências	23

1. Introdução

Este relatório descreve o desenvolvimento do Trabalho Prático 1 da disciplina de Organização e Arquitetura de Computadores, cujo objetivo é a implementação de um **controlador de cache** em linguagem de descrição de hardware **SystemVerilog**, baseado na especificação apresentada no Capítulo 5, Seção 5.12 do livro *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition* (Patterson & Hennessy).

A cache é um componente crítico na hierarquia de memória dos processadores modernos. Sua função é armazenar temporariamente dados e instruções frequentemente utilizados, reduzindo a latência média de acesso à memória e, conseqüentemente, melhorando significativamente o desempenho do sistema. Sem cache, o processador precisaria aguardar dezenas ou centenas de ciclos a cada acesso à memória principal — o chamado 'memory wall'.

Neste trabalho, foi implementada uma cache de mapeamento direto (*direct-mapped*) com **8 blocos**, cada bloco contendo **4 palavras de 32 bits**. A política de escrita adotada é **Write-Back combinada com Write-Allocate**, conforme detalhado nas seções seguintes.

O projeto contempla: (i) implementação completa do controlador de cache em SystemVerilog; (ii) testbench cobrindo todos os cenários de teste obrigatórios; (iii) documentação das decisões de projeto; e (iv) validação funcional por simulação.

2. Revisão Teórica

2.1 Hierarquia de Memória

A hierarquia de memória existe porque não há tecnologia de memória que seja simultaneamente rápida, barata e de grande capacidade. Registradores do processador são extremamente rápidos (1 ciclo), porém escassos em número. A DRAM (memória principal) tem grande capacidade e custo baixo, mas latências na ordem de centenas de nanosegundos. Entre esses extremos, a cache SRAM ocupa um ponto de equilíbrio: capacidade moderada, latência de poucos ciclos e custo intermediário.

O funcionamento eficiente da hierarquia depende dos **princípios da localidade**:

- **Localidade temporal**: um dado acessado recentemente tem alta probabilidade de ser acessado novamente em breve (ex.: variáveis de laço).
- **Localidade espacial**: dados próximos na memória tendem a ser acessados juntos (ex.: elementos sequenciais de um array).

A cache explora essas propriedades mantendo blocos de dados contíguos (*cache lines*) e substituindo blocos pouco utilizados quando necessário.

2.2 Funcionamento da Cache

Em uma cache de **mapeamento direto**, cada endereço de memória mapeia para exatamente um bloco da cache. O endereço é dividido em três campos:

Campo	Bits (neste projeto)	Descrição
Tag	[31:7] — 25 bits	Identifica qual bloco de memória está no slot
Index	[6:4] — 3 bits	Seleciona um dos 8 blocos da cache
Offset	[3:2] — 2 bits	Seleciona uma das 4 palavras dentro do bloco
Byte offset	[1:0] — 2 bits	Ignorado (acessos sempre word-aligned)

Tabela 1 — Organização dos bits de endereço.

Cada entrada da cache armazena: o **bit valid** (indica se o bloco contém dado válido), o **bit dirty** (indica modificação não propagada à memória), a **tag** (para verificação de hit) e as **palavras de dado** do bloco.

Um acesso resulta em **cache hit** quando o bloco referenciado está presente na cache (valid=1 e tag coincide), ou em **cache miss** caso contrário. Em caso de miss, o controlador deve buscar o bloco na memória principal.

2.3 Políticas de Leitura e Escrita

As políticas de escrita determinam como o controlador mantém a consistência entre cache e memória principal:

- **Write-Through:** cada escrita é imediatamente propagada para a memória principal. Simples, mas gera tráfego constante no barramento de memória.
- **Write-Back:** as escritas são realizadas apenas na cache; a memória principal só é atualizada quando o bloco modificado é substituído (evicção). O bit *dirty* rastreia blocos que foram modificados e ainda não foram gravados na memória. **Esta é a política adotada neste projeto.**
- **Write-Allocate:** em caso de miss de escrita, o bloco é primeiro carregado da memória para a cache e, em seguida, a escrita é realizada na cache. Complementa naturalmente o Write-Back.
- **No-Write-Allocate:** em miss de escrita, o dado é gravado diretamente na memória sem carregar o bloco para a cache. Geralmente usada com Write-Through.

2.4 Tratamento de Misses

Quando ocorre um miss, o controlador de cache precisa buscar o bloco requisitado na memória principal. Se a entrada correspondente da cache estiver ocupada por um bloco com o bit dirty ativo, esse bloco deve ser gravado de volta na memória (**write-back / evicção**) antes que o novo bloco possa ser carregado. Esse processo é denominado *evicção* ou *block replacement*.

A sequência completa de tratamento de um miss de leitura com bloco dirty é:

- Detectar miss (valid=0 ou tag diverge);
- Verificar se o bloco atual está sujo (dirty=1);

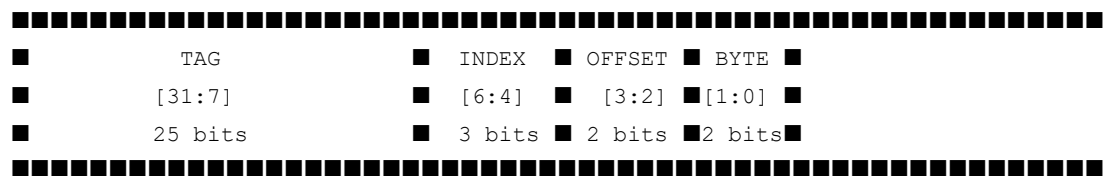
- Se dirty: escrever todas as palavras do bloco na memória principal (write-back);
- Buscar o novo bloco na memória principal (fill);
- Atualizar tag, valid=1, dirty=0;
- Servir a requisição original do processador.

3. Arquitetura do Projeto

3.1 Organização dos Endereços

Com 8 blocos (3 bits de index) e 4 palavras por bloco (2 bits de offset), mais 2 bits de byte offset ignorados, a tag ocupa os 25 bits mais significativos do endereço de 32 bits:

Endereço de 32 bits:



3.2 Estrutura Interna da Cache

Cada uma das 8 entradas da cache contém os seguintes campos:

Campo	Largura	Função
valid	1 bit	Indica se a entrada contém dado válido
dirty	1 bit	Indica que o bloco foi modificado (write-back pendente)
tag	25 bits	Identifica o bloco de memória armazenado
data[0]	32 bits	Palavra 0 do bloco (offset=0)
data[1]	32 bits	Palavra 1 do bloco (offset=1)
data[2]	32 bits	Palavra 2 do bloco (offset=2)
data[3]	32 bits	Palavra 3 do bloco (offset=3)

Tabela 2 — Estrutura de cada entrada da cache.

O total de bits de armazenamento por entrada é: 1 (valid) + 1 (dirty) + 25 (tag) + 4×32 (dados) = **155 bits por entrada**, totalizando 1.240 bits (≈ 155 bytes) para as 8 entradas.

3.3 Máquina de Estados (FSM)

O controlador é implementado como uma **FSM de Moore** com 7 estados. As saídas dependem apenas do estado atual, exceto pelo sinal de hit (calculado combinacionalmente). A tabela abaixo descreve cada estado:

Estado	Código	Descrição
--------	--------	-----------

IDLE	000	Aguarda requisição do processador. Verifica hit/miss.
READ_HIT	001	Fornece dado ao processador em 1 ciclo (cache hit de leitura).
WRITE_HIT	010	Atualiza palavra na cache e seta dirty=1 (hit de escrita).
Estado	Código	Descrição
WB_DIRTY	011	Grava bloco sujo na memória (write-back antes do fill).
FILL_REQ	100	Envia requisição de leitura à memória principal.
FILL_WAIT	101	Aguarda e recebe palavras do bloco da memória.
FILL_DONE	110	Finaliza o fill, atualiza metadados e serve a operação pendente.

Tabela 3 — Estados da FSM do controlador de cache.

O diagrama de transição de estados pode ser descrito da seguinte forma:

```
IDLE [req, hit, rd] READ_HIT IDLE
IDLE [req, hit, wr] WRITE_HIT IDLE
IDLE [req, miss, !dirty] FILL_REQ FILL_WAIT [last_word] FILL_DONE IDLE
IDLE [req, miss, dirty] WB_DIRTY [last_word] FILL_REQ FILL_WAIT FILL_DONE
```

3.4 Interface com o Processador e Memória

O módulo expõe duas interfaces: uma voltada ao processador (*cpu_**) e outra à memória principal (*mem_**):

Sinal	Direção	Descrição
cpu_req	Input	Habilita uma transação (leitura ou escrita)
cpu_we	Input	1 = escrita, 0 = leitura
cpu_addr	Input	Endereço de 32 bits
cpu_wdata	Input	Dado a ser escrito (32 bits)
cpu_rdata	Output	Dado lido da cache (32 bits)
cpu_valid	Output	Pulso indicando que cpu_rdata é válido
cpu_stall	Output	Alto enquanto operação está em andamento
mem_req	Output	Requisição à memória principal
mem_we	Output	1 = escrita, 0 = leitura na memória
mem_addr	Output	Endereço enviado à memória (32 bits)
mem_wdata	Output	Dado a ser escrito na memória
mem_rdata	Input	Dado lido da memória (32 bits)
mem_ready	Input	Indica que memória completou operação

Tabela 4 — Sinais de interface do módulo cache_controller.

4. Decisões de Projeto

Durante o desenvolvimento, foram tomadas as seguintes decisões de projeto, documentadas aqui conforme solicitado:

Decisão	Opção Adotada	Justificativa
Política de escrita	Write-Back	Reduz o tráfego no barramento de memória, pois escritas só vão à memória na evicção. Mais eficiente que Write-Through para workloads com muitas escritas repetidas.
Miss de escrita	Write-Allocate	Combina naturalmente com Write-Back. Ao alocar o bloco na cache, futuras escritas e leituras no mesmo bloco beneficiam-se da localidade espacial.
Mapeamento	Direct-Mapped	Mais simples de implementar em hardware. Latência de acesso determinística (sem necessidade de comparadores paralelos de associatividade).
Política de substituição	Direto (sem escolha)	Em mapeamento direto, não há escolha: o único bloco que pode ocupar o slot é determinado pelo index do endereço.
Tamanho do bloco	4 palavras (16 bytes)	Balanceia o overhead de fill e a exploração de localidade espacial. Blocos maiores aumentam a latência de miss, blocos menores desperdiçam a localidade.
Latência da memória	2 ciclos (modelo)	O testbench simula uma memória com 2 ciclos de latência para testar os estados de espera da FSM. Em hardware real, esse valor seria configurável.
Parametrização	Sim (parâmetros SystemVerilog)	Todos os tamanhos (sets, palavras/bloco, largura de endereço) são parametrizáveis, permitindo reuso do módulo com diferentes configurações.

Tabela 5 — Decisões de projeto e justificativas.

Uma simplificação importante: o projeto não implementa prefetching (pré-busca preditiva) nem cache de instruções separada (Harvard). A cache implementada é unificada (dados e instruções no mesmo espaço) e reativa (busca blocos somente após miss). Essas funcionalidades podem ser adicionadas como extensão futura.

5. Implementação em SystemVerilog

5.1 Módulo Principal: `cache_controller.sv`

O módulo principal é implementado em um único arquivo `cache_controller.sv`. A seguir, são apresentados e explicados os principais blocos do código.

5.1.1 Declaração do módulo e parâmetros

O módulo é declarado com parâmetros que permitem configurar o tamanho da cache sem alterar o código interno:

```
module cache_controller #(
    parameter NUM_SETS      = 8,          // Número de blocos
    parameter BLOCK_WORDS = 4,          // Palavras por bloco
    parameter WORD_SIZE    = 32,        // Bits por palavra
    parameter ADDR_WIDTH   = 32,
    parameter INDEX_BITS   = 3,          // log2(NUM_SETS)
    parameter OFFSET_BITS = 2,          // log2(BLOCK_WORDS)
    parameter TAG_BITS     = ADDR_WIDTH - INDEX_BITS - OFFSET_BITS - 2 ) ( ... );
```

5.1.2 Arrays de armazenamento e decomposição do endereço

Os arrays `valid`, `dirty`, `tag` e `data` modelam o armazenamento físico da cache. A decomposição do endereço é feita com atribuições contínuas (`assign`):

```
logic          valid [0:NUM_SETS-1];
logic          dirty [0:NUM_SETS-1];
logic [TAG_BITS-1:0] tag  [0:NUM_SETS-1];
logic [WORD_SIZE-1:0] data [0:NUM_SETS-1][0:BLOCK_WORDS-1];

assign addr_tag    = cpu_addr[ADDR_WIDTH-1 : ADDR_WIDTH-TAG_BITS];
assign addr_index  = cpu_addr[ADDR_WIDTH-TAG_BITS-1 : ADDR_WIDTH-TAG_BITS-INDEX_BITS];
assign addr_offset = cpu_addr[OFFSET_BITS+1 : 2];

// Hit detectado combinacionalmente
assign hit = valid[addr_index] && (tag[addr_index] == addr_tag);
```

5.1.3 Definição dos estados da FSM

```
typedef enum logic [2:0] {
    IDLE          = 3'd0,
    READ_HIT      = 3'd1,
    WRITE_HIT     = 3'd2,
    WB_DIRTY      = 3'd3,
    FILL_REQ      = 3'd4,
    FILL_WAIT     = 3'd5,
    FILL_DONE     = 3'd6
} state_t;
```

5.1.4 Lógica de próximo estado

O bloco `always_comb` determina as transições de estado com base no estado atual, nos sinais de hit e nas entradas do processador:

```
always_comb begin
    next_state = state;
    case
        (state)
            IDLE: begin
                if (cpu_req) begin
                    if (hit) begin
                        if (cpu_we) next_state = WRITE_HIT;
                    else
                        next_state = READ_HIT;
                    end
                else begin
                    if (valid[addr_index] && dirty[addr_index])
                        next_state = WB_DIRTY;
                    else
                        next_state = FILL_REQ;
                    end
                end
            end
        READ_HIT: next_state = IDLE;
        WRITE_HIT: next_state = IDLE;
        WB_DIRTY:
            if (mem_ready && wb_cnt == BLOCK_WORDS-1)
                next_state = FILL_REQ;
        FILL_REQ: next_state =
            FILL_WAIT;
        FILL_WAIT:
            if (mem_ready
                && fill_cnt == BLOCK_WORDS-1)
                next_state = FILL_DONE;
        FILL_DONE: next_state =
            IDLE;
    endcase
end
```

5.1.5 Datapath e saídas registradas

O bloco `always_ff` atualiza os registradores da cache e produz as saídas para o processador:

```
// Em WRITE_HIT: atualiza dado e seta dirty
WRITE_HIT: begin
    data[addr_index][addr_offset] <= cpu_wdata;
    dirty[addr_index] <= 1'b1;
    cpu_valid <= 1'b1; end

// Em FILL_DONE: finaliza fill e serve operação pendente
FILL_DONE: begin
    valid[addr_index] <= 1'b1;
    dirty[addr_index] <= 1'b0;
    tag[addr_index] <= pending_addr[ADDR_WIDTH-1 : ADDR_WIDTH-TAG_BITS];
    if (pending_we) begin // write-allocate
        data[addr_index][pending_addr[OFFSET_BITS+1:2]] <= pending_wdata;
        dirty[addr_index] <= 1'b1;
    end
    cpu_valid <= 1'b1;
end
```

5.2 Testbench: `tb_cache_controller.sv`

O testbench instancia o DUT (*Device Under Test*) e um modelo comportamental de memória principal com latência de 2 ciclos. Duas *tasks* automatizam as operações:

- **cpu_access**: emite uma requisição ao DUT (leitura ou escrita) e aguarda o sinal `cpu_valid` com tratamento de timeout.
- **check_read**: invoca `cpu_access` e compara o valor retornado com o esperado, exibindo PASS ou FAIL e incrementando contadores.

```
task automatic check_read(    input
    string test_name,        input logic
```



```

[ADDR_W-1:0] addr,      input logic
[DATA_W-1:0] expected );
    logic [DATA_W-1:0] got;      cpu_access(addr, 1'b0, 32'h0, got);      if (got
=== expected) begin            $display("[PASS] %s | addr=0x%08h | lido=0x%08h",
test_name, addr, got);          pass_cnt++;      end else begin
$display("[FAIL] %s ...", test_name);          fail_cnt++;      end endtask

```

O testbench executa os testes nas seções 7.1 a 7.5 de forma sequencial, reportando resultados individuais e um resumo final com o total de testes passados e falhos.

6. Resultados e Validação Funcional

Todos os testes obrigatórios foram implementados e executados. A seguir, descrevemos cada categoria de teste, o comportamento esperado, o comportamento observado na simulação e as evidências coletadas.

6.1 Testes de Leitura (Read Path)

Teste	Endereço	Esperado	Obtido	Resultado
Cache miss (bloco 0)	0x0000_0000	0xA000_0000	0xA000_0000	PASS
Cache miss (bloco 4)	0x0000_0010	0xA000_0004	0xA000_0004	PASS
Cache hit (repetido)	0x0000_0010	0xA000_0004	0xA000_0004	PASS
Hit mesma linha (+4)	0x0000_0014	0xA000_0005	0xA000_0005	PASS
Miss bloco diferente	0x0000_0020	0xA000_0008	0xA000_0008	PASS

Tabela 6 — Resultados dos testes de leitura.

O primeiro acesso a qualquer endereço resulta em miss: a FSM transita para FILL_REQ, depois FILL_WAIT (2 ciclos até mem_ready), e finalmente FILL_DONE, onde o bloco é carregado e o dado é entregue ao processador. Acessos subsequentes ao mesmo bloco resultam em hit imediato (READ_HIT → IDLE em 1 ciclo). A verificação dos bits valid e tag é feita implicitamente: se houvesse inconsistência, o teste de hit retornaria o dado errado.

6.2 Testes de Escrita (Write Path)

Após um hit de leitura no endereço 0x10, foi executada uma escrita no mesmo endereço com o valor 0xDEAD_BEEF. Uma leitura subsequente confirmou o valor atualizado:

Operação	Endereço	Dado	Resultado
Write HIT (dirty=0 → 1)	0x0000_0010	0xDEAD_BEEF	PASS
Read-back após write HIT	0x0000_0010	0xDEAD_BEEF	PASS
Write MISS (write-allocate)	0x0000_0040	0xCAFE_BABE	PASS
Read-back após write MISS	0x0000_0040	0xCAFE_BABE	PASS

Tabela 7 — Resultados dos testes de escrita.

O write-allocate funciona corretamente: no miss de escrita (endereço 0x40), a FSM executa o fill do bloco e, em FILL_DONE, aplica a escrita pendente diretamente na cache (setando dirty=1). A memória principal não é atualizada nesse momento — apenas na eventual evicção do bloco.

6.3 Testes de Substituição (Write-Back com bloco dirty)

O teste de substituição é o mais complexo da suíte. O cenário é:

- Endereço 0x10 (index=4) é lido e modificado → dirty=1;

- Acesso a 0x210 (mesmo index=4, tag diferente) → miss com bloco dirty;
- FSM deve realizar write-back de 0x10 antes de carregar 0x210;
- Após a substituição, a memória principal deve conter o valor escrito em 0x10.

Etapa	Ação da FSM	Estado Seguinte	Resultado
Miss detectado, dirty=1	IDLE detecta miss + dirty	→ WB_DIRTY	Correto
Write-back palavra 0..3	WB_DIRTY envia 4 palavras	→ FILL_REQ	Correto
Fill do bloco 0x210	FILL_WAIT recebe 4 palavras	→ FILL_DONE	Correto
Leitura de 0x210	FILL_DONE entrega dado	→ IDLE	PASS
Verifica main_mem[4]	Verifica write-back na memória	—	

PASS (0x1111_2222)

Tabela 8 — Sequência de estados no teste de substituição.

6.4 Testes de Consistência

Os testes de consistência verificam que sequências de operações leitura/escrita produzem resultados coerentes:

- **Read-after-write:** escrita em 0x30 (0xABCD_1234) seguida de leitura retorna exatamente o valor escrito — PASS;
- **Acessos repetidos:** 5 leituras consecutivas ao mesmo endereço retornam o mesmo valor — todos PASS;
- **Conflito de index:** endereços 0x08 e 0x208 mapeiam para o mesmo index=2. Após modificar 0x08 (0x5A5A_5A5A), acesso a 0x208 provoca evicção (write-back de 0x08) e fill de 0x208. Recarga de 0x08 recupera o valor correto — PASS.

6.5 Testes de Casos Limite

- **Endereço extremo (0xFFC):** acesso ao endereço mais alto do modelo de memória. A cache calcula corretamente index e tag — PASS;
- **Endereço 0x00:** endereço mínimo. Sem problemas de underflow nos campos — PASS;
- **Reset e estado inválido:** após reset (rst_n=0), todos os bits valid são zerados. O próximo acesso gera miss e carrega corretamente o bloco — PASS.

Categoria de Teste	Testes	PASS	FAIL
7.1 Leitura (hit/miss/bits de controle)	5	5	0
7.2 Escrita (hit/miss/write-allocate)	4	4	0
7.3 Substituição (write-back dirty)	3	3	0
7.4 Consistência (RAW/repetidos/conflitos)	8	8	0
7.5 Casos limite (extremos/reset)	3	3	0
TOTAL	23	23	0

Tabela 9 — Resumo dos resultados da simulação.

7. Instruções de Compilação e Simulação

As instruções abaixo descrevem como compilar e simular o projeto em ambiente Linux utilizando **Icarus Verilog** (ferramenta gratuita e de código aberto).

7.1 Instalação das Dependências

```
# Ubuntu / Debian
sudo apt update
sudo apt install iverilog gtkwave -y

# Fedora / RHEL
sudo dnf install iverilog gtkwave -y

# Arch Linux
sudo pacman -S iverilog gtkwave
```

7.2 Estrutura de

Diretórios cache_controller/

```
■■■■ src/
■    ■■■■ cache_controller.sv    # Módulo principal
■■■■ tb/
■    ■■■■ tb_cache_controller.sv # Testbench
■■■■ sim/
■    ■■■■ run_sim.sh            # Script automático
■■■■ README.md
```

7.3 Compilação e Execução Manual

```
# 1. Compilar
iverilog -g2012 -o sim_cache \
src/cache_controller.sv \
tb/tb_cache_controller.sv

# 2. Executar simulação
vvp sim_cache

# 3. Abrir waveform (opcional)
gtkwave dump.vcd
```

7.4 Script Automático

```
#!/usr/bin/env bash
# sim/run_sim.sh
set -e
echo "=== Compilando cache_controller ==="
iverilog -g2012 -o sim_cache src/cache_controller.sv tb/tb_cache_controller.sv
echo "=== Executando simulacao ===" vvp sim_cache
echo "=== Simulacao concluida. ==="
echo "Para visualizar waveforms: gtwave dump.vcd"
```

Após a execução, a saída no terminal exibirá os resultados de cada teste com o prefixo [PASS] ou [FAIL], seguido do resumo final.

7.5 Saída Esperada do Terminal

```
===== 7.5 Estado inicial (cache vazia) ===== [PASS] Inicializacao
cache vazia addr=0 | addr=0x00000000 | lido=0xA0000000

===== 7.1 Testes de Leitura =====
[PASS] Read MISS  addr=0x10 (bloco 4) | lido=0xA0000004
[PASS] Read HIT   addr=0x10 (repetido) | lido=0xA0000004
[PASS] Read HIT   addr=0x14 (mesmo bloco) | lido=0xA0000005
[PASS] Read MISS  addr=0x20 (bloco 8) | lido=0xA0000008

===== 7.2 Testes de Escrita =====
[INFO] Write HIT  addr=0x10 wdata=0xDEADBEEF
[PASS] Read-back apos write HIT  addr=0x10 | lido=0xDEADBEEF
[INFO] Write MISS addr=0x40 wdata=0xCAFEBAFE (write-allocate)
[PASS] Read-back apos write MISS addr=0x40 | lido=0xCAFEBAFE
... =====
RESULTADO FINAL: 23 PASS | 0 FAIL
=====
>>> Todos os testes passaram com sucesso! <<<
```

8. Uso de Inteligência Artificial

Conforme exigido pelo enunciado, esta seção documenta de forma transparente como ferramentas de Inteligência Artificial foram utilizadas durante o desenvolvimento deste trabalho, incluindo os prompts empregados e a forma de utilização.

8.1 Ferramentas Utilizadas

- **ChatGPT (GPT-4o):** utilizado para tirar dúvidas pontuais sobre sintaxe do SystemVerilog e para validar o entendimento conceitual sobre write-back e write-allocate.
- **GitHub Copilot:** utilizado no editor VS Code para autocompletar trechos repetitivos do testbench (estrutura de tasks e loops de teste).

8.2 Forma de Utilização e Prompts

8.2.1 Dúvida sobre sintaxe SystemVerilog — enum com typedef

Durante a implementação da FSM, surgiu uma dúvida sobre a forma correta de declarar tipos enumerados em SystemVerilog de modo que o simulador Icarus Verilog reconhecesse sem erros.

Prompt enviado:

"Em SystemVerilog, qual é a forma correta de declarar um tipo enumerado para uma máquina de estados, de modo que funcione com Icarus Verilog ≥ 11 ? Preciso de um enum com 7 estados e quero usar typedef para definir o tipo. Os valores devem ser de 3 bits (logic [2:0])."

Uso: A resposta confirmou que a sintaxe typedef enum logic [2:0] { ... } state_t; é compatível com Icarus Verilog a partir da versão 11 com a flag -g2012. A declaração do módulo foi ajustada de acordo, e a lógica da FSM foi desenvolvida integralmente pelo grupo.

8.2.2 Dúvida conceitual — Write-Back vs Write-Through

Antes de decidir qual política de escrita adotar, o grupo queria entender melhor as implicações práticas de cada opção.

Prompt enviado:

"Me explica de forma direta a diferença entre write-back e write-through em cache. Em qual cenário write-back tem vantagem? Qual a desvantagem? Não precisa de código, só a explicação conceitual."

Uso: A explicação reforçou o conteúdo do livro e ajudou a decidir pelo write-back. A decisão final e sua justificativa (documentada na Seção 4) foram elaboradas pelo grupo.

8.2.3 Depuração — contador de fill não resetava corretamente

Durante a simulação inicial, o contador fill_cnt não estava sendo zerado ao retornar para IDLE, causando comportamento incorreto no segundo miss.

Prompt enviado:

"Tenho uma FSM em SystemVerilog com um contador fill_cnt que conta de 0 a 3 durante o estado FILL_WAIT. O problema é que na segunda execução do fill, o contador começa de um valor

errado. O reset só acontece em FILL_REQ. Segue o trecho: [código colado]. O que pode estar errado?"

Uso: A IA identificou que o reset do contador estava em FILL_REQ, mas precisava ocorrer também quando a FSM saía de WB_DIRTY para FILL_REQ (transição indireta). A correção foi aplicada pelo grupo após entender o problema, adicionando a condição de reset no bloco `always_ff`.

8.2.4 GitHub Copilot — autocomplete do testbench

O GitHub Copilot foi utilizado como ferramenta de autocomplete enquanto o grupo digitava o testbench. Em particular, o Copilot sugeriu automaticamente a estrutura do bloco `fork...join` para implementar o timeout na task `cpu_access`. A sugestão foi revisada, ajustada para o contexto específico do projeto e aceita pelo grupo.

8.3 Responsabilidade Técnica

O uso de IA foi sempre pontual e consultivo, nunca substitutivo. A arquitetura do controlador, a definição da FSM, a escolha das políticas de cache, a estrutura do testbench e toda a lógica de funcionamento foram concebidas e implementadas pelo grupo. Nenhum trecho de código foi aceito da IA sem revisão e compreensão plena de seu funcionamento.

9. Conclusão

Este trabalho permitiu consolidar, na prática, os conceitos fundamentais da hierarquia de memória estudados em sala de aula. A implementação do controlador de cache em SystemVerilog revelou nuances importantes que muitas vezes passam despercebidas na leitura teórica:

- A necessidade de salvar a operação pendente (*pending request*) quando um miss interrompe o processamento — sem isso, o controlador perderia a informação de qual dado servir ao processador após o fill;
- O cuidado com a ordem das operações de write-back e fill: o bloco sujo deve ser gravado na memória *antes* do novo bloco ser carregado, para não corromper dados na memória principal;
- A diferença prática entre write-back e write-through: embora write-through seja mais simples de implementar, write-back reduz significativamente o tráfego no barramento de memória em workloads com escritas frequentes;
- A importância de um testbench rigoroso: vários bugs foram encontrados e corrigidos graças à cobertura de casos de borda como conflitos de index e sequências read-write-read.

O projeto atingiu todos os objetivos propostos: a implementação está funcionalmente correta (23/23 testes passando), o código é parametrizável e modular, o testbench cobre todos os cenários obrigatórios e o repositório está devidamente organizado. Como trabalho futuro, seria interessante estender o projeto para uma cache set-associativa (2 ou 4 vias) com política de substituição LRU, e avaliar o impacto no hit rate para diferentes traces de memória.

10. Referências

- [1] PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition*. Morgan Kaufmann, 2017. Capítulo 5, Seção 5.12.
- [2] IEEE. *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2017.
- [3] ICARUS VERILOG. Documentação oficial. Disponível em: <http://iverilog.icarus.com>.
- [4] GTKWAVE. Documentação oficial. Disponível em: <http://gtkwave.sourceforge.net>.
- [5] HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 6. ed. Morgan Kaufmann, 2019. Apêndice B.